



Compte Rendu de Stage — Séance 3

KEAR5AAU – Automatique des Systèmes Linéaires

Étudiant : A. HADJOUDJ
Encadrant : C. Louembet
Date : 9 juin 2026

Année universitaire 2025–2026

Objectif

L'objectif est de développer un second script Python générant des questions pédagogiques portant sur l'analyse de stabilité des systèmes linéaires, pour les ordres 1 à 4, au format XML compatible avec Moodle.

Rappel théorique — Stabilité des systèmes linéaires

Un système linéaire est **stable** si tous ses pôles ont une partie réelle strictement négative.

Condition de stabilité par ordre

- **Ordre 1** : $H(p) = \frac{1}{p+a}$ — stable si $a > 0$
- **Ordre 2** : $H(p) = \frac{1}{p^2+ap+b}$ — stable si $a > 0$ et $b > 0$
- **Ordre 3** : $H(p) = \frac{1}{p^3+ap^2+bp+c}$ — critère de Routh : $a > 0$, $c > 0$ et $ab > c$
- **Ordre 4** : $H(p) = \frac{1}{p^4+ap^3+bp^2+cp+d}$ — critère de Routh : $a > 0$, $d > 0$, $(ab-c) > 0$ et $(abc-a^2d-c^2) > 0$

Cas particulier Pôle = 0

On peut pas savoir si un système est stable lorsqu'un ou plusieurs coefficients sont nuls. Le système n'est ni stable ni instable.

Exemple illustratif — Contre-exemple ordre 3

Soit $H(p) = \frac{1}{p^3+p^2+p+10}$ avec $a = 1$, $b = 1$, $c = 10$.

Vérification du critère de Routh :

- $a > 0$
- $c > 0$
- $ab > c : 1 \times 1 = 1 > 10$

Conclusion : Le système est instable malgré des coefficients positifs. C'est pourquoi la condition $ab > c$ est indispensable.

Développement du script Python

Approche de développement

Le script a été développé progressivement, ordre par ordre, en vérifiant à chaque étape que les conditions de stabilité étaient correctement implémentées.

Nouvelles notions Python apprises

- **Structure if/elif/else** — pour gérer les différents cas (stable, instable, on ne peut pas savoir)
- **Opérateur and** — pour combiner plusieurs conditions
- **Opérateur **** — pour les puissances (ex : $a^{**2} = a^2$)
- `random.randint(-10, 10)` — pour générer des coefficients positifs et négatifs

Ordre 1

```

1 for i in range(5):
2     choix = random.randint(1, 3)
3     if choix == 1:
4         a = random.randint(1, 10)          # stable
5         bonne = 'Oui, car le pole est a partie reelle negative'
6         mauvaise = 'Non, car le pole est a partie reelle positive'
7     elif choix == 2:
8         a = random.randint(-10, -1)       # instable
9         bonne = 'Non, car le pole est a partie reelle positive'
10        mauvaise = 'Oui, car le pole est a partie reelle negative'
11    else:
12        a = 0                             # on ne peut pas savoir
13        bonne = 'on ne peut pas savoir'
14        mauvaise = 'Oui, car le pole est a partie reelle negative'

```

Problème rencontré : Au départ, `a` était généré avec `randint(-5, 5)`, ce qui incluait 0. Cela créait un cas non traité par le `if/elif`.

Solution : Utilisation d'un choix aléatoire entre 3 cas distincts, chacun associé à une plage de valeurs spécifique.

Ordre 2

```

1 for i in range(5):
2     a = random.randint(-10, 10)
3     b = random.randint(-10, 10)
4     if a > 0 and b > 0:
5         bonne = 'Oui, car le pole est a partie reelle negative'
6         mauvaise = 'Non, car le pole est a partie reelle positive'
7     elif a == 0 or b == 0:
8         bonne = 'On ne peut pas savoir'
9         mauvaise = 'Oui, car le pole est a partie reelle negative'
10    else:
11        bonne = 'Non, car le pole est a partie reelle positive'
12        mauvaise = 'Oui, car le pole est a partie reelle negative'

```

Amélioration apportée : Le code a été optimisé en utilisant directement les conditions `a > 0 and b > 0` au lieu de gérer 4 cas séparés (`a+/b+`, `a-/b-`, `a+/b-`, `a-/b+`). Cela rend le code plus lisible et plus maintenable.

Ordre 3 — Critère de Routh

```

1 for i in range(5):
2     a = random.randint(-10, 10)
3     b = random.randint(-10, 10)
4     c = random.randint(-10, 10)
5     if (a > 0 and c > 0 and (a*b - c) > 0):
6         bonne = 'Oui, car le pole est a partie reelle negative'
7         mauvaise = 'Non, car le pole est a partie reelle positive'
8     elif a == 0 or b == 0 or c == 0:
9         bonne = 'On ne peut pas savoir'
10        mauvaise = 'Oui, car le pole est a partie reelle negative'
11    else:
12        bonne = 'Non, car le pole est a partie reelle positive'
13        mauvaise = 'Oui, car le pole est a partie reelle negative'

```

Problème rencontré : La condition initiale $a > 0$ and $b > 0$ and $c > 0$ était incorrecte — elle ne suffisait pas pour garantir la stabilité.

Solution : Après analyse, la condition correcte du critère de Routh pour l'ordre 3 est $a > 0$, $c > 0$ et $ab > c$, soit $(a*b - c) > 0$ en Python.

Ordre 4 — Critère de Routh complet

```

1 for i in range(5):
2     a = random.randint(-10, 10)
3     b = random.randint(-10, 10)
4     c = random.randint(-10, 10)
5     d = random.randint(-10, 10)
6     if (a > 0 and d > 0 and (a*b - c) > 0
7         and (a*b*c - a*a*d - c*c) > 0):
8         bonne = 'Oui, car le pole est a partie reelle negative'
9         mauvaise = 'Non, car le pole est a partie reelle positive'
10    elif a == 0 or b == 0 or c == 0 or d == 0:
11        bonne = 'On ne peut pas savoir'
12        mauvaise = 'Oui, car le pole est a partie reelle negative'
13    else:
14        bonne = 'Non, car le pole est a partie reelle positive'
15        mauvaise = 'Oui, car le pole est a partie reelle negative'

```

Problème rencontré : La condition de Routh pour l'ordre 4 est plus complexe. La condition $abc > a^2d + c^2$ seule n'est pas suffisante — il faut également vérifier $ab > c$.

Solution : Les deux conditions ont été combinées dans le même if.

Améliorations apportées au cours du développement

Mélange des réponses

Au départ la bonne réponse apparaissait toujours en première position. Deux solutions ont été envisagées :

— **Solution Python :** utiliser `random.shuffle()` pour mélanger les réponses

- **Solution Moodle** : utiliser la balise `<shuffleanswers>1</shuffleanswers>` dans le XML

La solution Moodle a été retenue car plus simple et plus efficace — Moodle mélange les réponses automatiquement à chaque affichage de la question.

Compteur automatique de questions

Un compteur `total` a été ajouté pour afficher automatiquement le nombre de questions générées, sans avoir à le calculer manuellement.

```
1 total = 0
2 # dans chaque boucle :
3 total += 1
4 # a la fin :
5 print("Total : " + str(total) + " questions generees !")
```

Problème identifié — À corriger

Problème : Avec `randint(-10, 10)`, les conditions de Routh sont rarement satisfaites simultanément. En pratique, presque aucune question stable n'est générée.

Solution prévue : Forcer 50% de cas stables en générant d'abord un cas stable garanti, puis un cas instable, en alternance.

Résultats

Ordre	Condition de stabilité	Questions générées
Ordre 1	$a > 0$	5 questions
Ordre 2	$a > 0$ et $b > 0$	5 questions
Ordre 3	$a > 0, c > 0, ab > c$	5 questions
Ordre 4	$a > 0, d > 0, ab > c, abc > a^2d + c^2$	5 questions
Total		20 questions

Le nombre de questions est paramétrable en modifiant `range(5)`.

Prochaines étapes

1. Corriger le problème des cas stables — garantir 50% stable / 50% instable
2. Fusionner les deux scripts ((Eq Diff-FT) + Stabilité) en un seul fichier